

JSB Data Engineer

Practical Exercises — Candidate Submission

Payment reconciliation · Incremental API ingestion · Database design & reporting model

Tools used, end to end

MySQL 8.0 · MariaDB 10.11 (cross-verified) · Python 3.11 · dbt-core 1.7 (dbt-mysql) · Postman / Newman · Mermaid · Power BI (data model) · Databricks (Delta Lake, serverless)

Every number in this document was produced by actually running the code and queries shown — not hand-typed. Evidence for each figure lives in the accompanying repository.

How to read this document

This document is the write-up for all three practical exercises. Each section states the scenario briefly, the approach taken, the tools used, and the evidence that it actually works — screenshots are captures of real runs, not mock-ups. Full source code, SQL, and raw evidence files accompany this document in the submitted repository (folder: candidate-assessment/).

A short note on assumptions: per the brief's instruction not to stop on ambiguity, every assumption made is written down at the point it's used, not buried — for example the definition of NGR used in Exercise 3, and the fee-rounding tolerance used in Exercise 1.

Contents

- Exercise 1 — Payment Gateway Reconciliation
- Exercise 2 — Incremental, Restartable API Ingestion
- Exercise 3 — Database Design: Players, Wallets, Bets, Bonuses
- Power BI — Reporting Data Model and Report
- Databricks — The Same Three Exercises on Delta Lake

Exercise 1 — Payment Gateway Reconciliation

Finance Manager summary

Week 1–7 September 2026 (UTC) · NAD · internal_deposits (326 rows) vs gateway_settlement (306 rows)

274 of 306 gateway settlements (90%) match exactly on reference, gross, fee and net. The 49 differences are 34 genuine breaks, 6 timing differences and 9 that are not a problem (exceptions.csv). The bridge from our R218,280.00 to the gateway's R217,979.97 leaves R0.00 unexplained; its two timing lines are provisional (see assumptions).

Bridge on gross amounts: internal total → gateway total

Line	NAD
Internal SUCCESS deposits, total	218,280.00
– Duplicate internal deposit rows (same settlement counted twice)	(1,300.00)
– Reversals / chargebacks (not in the gateway's SETTLED total)	(850.00)
– Deposits with no settlement yet (mid-week)	(2,700.00)
– Created in the last 15 min, not yet settled (provisional)	(3,650.00)
+ Settled in the first 15 min, no deposit this week (provisional)	2,600.00
+ Settlements with no internal record	2,900.00
+ Settled by the gateway but marked FAILED by us	250.00
+ Duplicate gateway settlement rows	1,550.00
+/- Gross amount differences with the gateway (net)	900.00
+/- Rounding (exactly 1 cent on 3 rows)	(0.03)
= Gateway SETTLED total	217,979.97
Residual (unexplained)	0.00

Priority actions

When	Action	Owner	NAD
Today	Re-credit 2 players: marked FAILED by us, settled by the gateway	Payments	250.00
Today	Get gateway detail for 4 unrecognised settlements (GW-000301 to 304)	Payments	2,900.00
Week	Dispute 4 gross differences (3 players under-credited R950, 1 over R50)	Finance	900.00 net
Week	Claim fee overcharges (4) and short-paid nets (2)	Finance	7.75
Week	Did 2 duplicate settlements pay twice? Audit 3 duplicate deposits	Payments	1,550 / 1,300
Week	Confirm the wallet debit for each of 3 reversals	Finance	850.00
3 days	5 deposits with no settlement: re-check next file, then escalate	Finance	2,700.00

Material assumptions

- Timing lines are provisional: this week's files can't prove them. R3,650 (3 deposits created 23:54–23:55 on 7 Sep) should appear in the 8–14 Sep gateway file; R2,600 (3 settlements at 00:04–00:06 on 1 Sep, GW-000331 to 333) should match the 25–31 Aug internal file. Finance keeps all six open until matched; any unmatched item becomes a break.
- Fee = 2% of gross + R1.00, to the cent; any fee or net difference of 1 cent or more is flagged. Gross differences of exactly 1 cent (3 rows, R0.03) are treated as rounding, a materiality choice for Finance to confirm. Fee and net errors don't move gross, so they are actions, not bridge lines.

Supporting detail

Tools used: MySQL 8.0 for the schema, load and categorisation SQL; Python/pandas for the load script, CSV export and an independent cross-check of the bridge arithmetic.

Approach

Both source files were loaded as-is into MySQL. Matching between `internal_deposits.gateway_ref` and `gateway_settlement.merchant_ref` uses a normalised reference (upper-case, letters and digits only), because the gateway returns six of our references with different punctuation, case or spacing ("GW_000020", "GW000102", "gw-000196 ", " GW-000267 " and others). An exact-match join would misclassify those six clean matches as breaks.

Every internal SUCCESS deposit and every gateway settlement row is either a clean match or one of twelve exception types: a genuine break, a timing difference, a business event (reversal), or not a problem at all. The rules live in `exercise1-reconciliation/sql/03_reconciliation.sql` and are ported into a dbt model (`fct_recon_exceptions`) so they run on a schedule with tests attached. The checks follow the brief's contract exactly: fee = 2% of gross + R1.00 to the cent, and net = gross – fee. Boundary tests at 1, 2 and 3 cents (`sql/06_threshold_fixtures.py`) confirm that every fee or net difference of a cent or more is flagged.

```
mysql -- jsb_assessment -- reconciliation output

$ mysql -u assess -p jsb_assessment < sql/03_reconciliation.sql

category                                     n impact_total
OK: matched, amount and fee correct          274      0.00
BREAK: duplicate internal SUCCESS deposit for one settlement (double-credit risk) 6      0.00
BREAK: deposit SUCCESS, no gateway settlement found      5    2700.00
BREAK: settled fee differs from contracted fee           4       3.75
BREAK: settled gross amount differs from internal amount 4    -900.00
BREAK: duplicate gateway settlement for one reference (double-credit risk) 4      0.00
BREAK: unrecognised settlement (no internal record)      4    2900.00
NOT A PROBLEM: rounding difference <= 1 cent           3       0.03
REVERSAL: gateway reversed/charged back after settlement 3     850.00
TIMING: settlement expected in next period (created near cut-off) 3    3650.00
TIMING: prior-period deposit settled at start of period  3    2600.00
BREAK: net amount is not gross minus fee               2       4.00
BREAK: payment confirmed, wallet not credited           2    250.00

-- The bridge --
internal_success_total    218200.00
gateway_settled_total     217979.97
residual (unexplained)    0.00  <- every cent is accounted for

$ python sql/04_independent_check.py
pandas rows: 317  SQL rows: 317
AGREE on every row
```

Actual output of `sql/03_reconciliation.sql` against MySQL 8.0, and of the independent row-by-row check.

Exceptions by category

Every one of the 49 rows in `exceptions.csv` carries exactly one classification, as the brief asks: genuine break (34), timing difference (6) or not a problem (9), plus a row-specific explanation, next step and owner. `Reconciliation_Workbook.xlsx` holds the same rows and the two raw files, with the bridge and category totals as live SUMIFS/COUNTIFS formulas that recalculate to a 0.00 residual.

Category	Classification	Row s	Impact (NAD)
Payment confirmed, wallet not credited	Genuine break — act today	2	250.00 owed
Unrecognised settlement (no internal record)	Genuine break — act today	4	2,900.00
Net amount is not gross minus fee	Genuine break — dispute	2	4.00 short-paid

Category	Classification	Rows	Impact (NAD)
Settled fee differs from contract	Genuine break — dispute	4	3.75 overcharged
Settled amount differs from internal amount	Genuine break — dispute	4	-900.00 (net)
Duplicate gateway settlement	Genuine break — dispute	4	1,550.00*
Duplicate internal deposit	Genuine break — reverse credit	6	1,300.00*
Reversal / chargeback	Genuine break — claw back	3	850.00
Deposit SUCCESS, no settlement found	Genuine break — chase	5	2,700.00
Created in the last 15 min (expected next week)	Timing difference	3	3,650.00
Settled in the first 15 min (likely last week's)	Timing difference	3	2,600.00
Rounding, exactly 1 cent	Not a problem	3	0.03
Reference formatting differs (matched)	Not a problem	6	0.00

* Amount carried by the duplicate row only; the original row in each pair is 0. Only SUCCESS deposits credit a wallet, so each duplicate internal deposit credited the player twice for one payment. A dbt test (`assert_recon_bridge_reconciles`) fails the build if the bridge residual is ever not exactly zero.

Assumptions

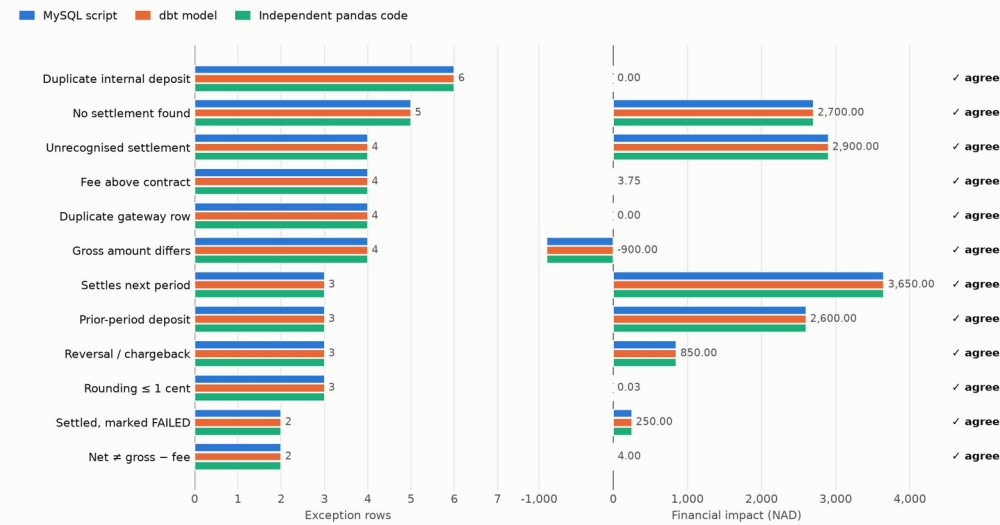
- Period 2026-09-01 00:00:00 to 2026-09-07 23:59:59 UTC inclusive; all times UTC; both files NAD only. Internal total = SUCCESS deposits; gateway total = SETTLED rows.
- References match after upper-casing and keeping only letters and digits. Fee = $\text{ROUND}(2\% \times \text{gross} + 1.00, 2)$; any fee or net difference of 1 cent or more is a break; a gross difference of exactly 1 cent is rounding (Finance to confirm).
- Cut-off window of 15 minutes at each end. Settlements arrive 28 min after the deposit on median, 50 min at most, and the nearest unexplained items are 2 hours or more from either boundary, so the result doesn't depend on the window. The six timing items stay provisional until matched.
- Reversals are genuine breaks until the wallet debit is confirmed. For duplicates, the later row is the duplicate. The bridge is on gross, so fee and net errors are actions, not bridge lines. FAILED deposits with no settlement agree on both sides and aren't listed.

How the figures were checked

The categorisation was implemented three times: the MySQL script, the dbt model, and separately written pandas code (`sql/04_independent_check.py`) that works per deposit and per settlement instead of through a join. All three agree on all 317 rows individually, and on every category's count and rand total.

Three separate implementations agree on every reconciliation category

Rows and rand impact per exception category. Clean matches: 274 · 274 · 274. All 317 rows also agree one-by-one.



Three implementations, identical results in every category (sql/05_agreement_chart.py).

That check was added after an audit of an earlier draft found real problems, all now fixed. The draft had no test for net = gross – fee, which the brief requires, so it missed two short-paid settlements. It didn't quantify the fee overcharge. It counted four reference-formatting variants instead of six, because a case-insensitive database comparison hid three. It filed three start-of-week settlements as unrecognised money rather than timing. A later review by Codex found the fee check tolerated differences of up to 2 cents and a 2-cent gross difference could be labelled rounding; the rules are now exact, with boundary tests, and no figure changed. The bridge balanced throughout. The errors were in how the differences were labelled and summarised, which is why the categories are now cross-checked independently.

Automating this daily

- Land both feeds daily as-is, never overwriting, and log control totals (row count, sum) per source before matching. A source-declared-vs-received mismatch is itself an alert.
- Run the categorisation as a scheduled dbt model; persist history, never overwrite yesterday's result.
- Alert tiers: page immediately on "wallet not credited" or "unrecognised settlement"; same-day on new duplicates or disputes above a materiality threshold; daily digest for reversals and timing items still within the cut-off window.
- Ageing: unresolved after 3 days escalates to Finance management; unresolved after 7 days requires a signed-off write-off or credit note.

Exercise 2 — Incremental, Restartable API Ingestion

Tools used: Python 3 (standard library HTTP, plus pymysql), MySQL 8 / MariaDB for the target and control tables (the brief allows any database; this one also holds the other exercises and dbt), Postman/Newman for the API contract.

Design

- Progress: a checkpoint table holds the last (updated_at, id) fully committed. Each run resumes with updated_since = that time (inclusive, per the API doc) and pages with the server's cursor. The API moves updated_at forward when a record changes, so one position covers new and changed records.
- No duplicates: id is the primary key and every write is an upsert on it. Repeated rows within a page are collapsed; a record is written only if it is new or its updated_at moved forward, so an older version never overwrites a newer one. A page's rows, its rejects, the checkpoint and the run counters commit in one transaction, so a kill loses at most the uncommitted page, which the next run re-reads.
- Failures: 429 waits for Retry-After; 500 and network errors retry with 2, 4, 8... s backoff, then the run ends FAILED with exit code 1 and the checkpoint unchanged. A database lock stops two scheduled runs overlapping; a run killed mid-way is marked ABANDONED by the next.
- Unclean data: numbers sent as strings are coerced; records with a missing player_id, an unparseable amount or time, or a negative amount go to ingest_rejects with the reason and raw JSON, once per distinct payload. The run carries on.
- Monitoring: ingest_runs records, per run, pages, new / changed / unchanged / rejected rows, 429 and 500 counts, start and end time, status and any error; the checkpoint is the last successful position. checks.sql holds the monitoring and alert queries.

Evidence: kill, restart, new activity

demo.py reproduces the whole sequence on any machine, with the mock API's faults switched on. Run 1 uses 50-record pages and a pause before each commit, and is hard-killed while page 2 is uncommitted. Only page 1 (50 rows) is in the table; the restart marks run 1 ABANDONED, resumes from the checkpoint and loads the rest.

```

Exercise 2: hard kill mid-page, then restart

$ python demo.py (steps 3-6: kill mid-page, restart, check against the API)
3. Run 1: 50-record pages, 2 s pause before each commit; kill it mid-page
| == run 1 starting ==
| no checkpoint found: full initial load
| [demo] sleeping 2.0s before commit (kill now to test safety)
| page 1: 50 new, 0 changed, 0 unchanged, 0 rejected, has_more=True
| [demo] sleeping 2.0s before commit (kill now to test safety)
>>> process hard-killed mid-page (page 2 fetched, not committed)

4. State right after the kill: only page 1 is in the table
rows_loaded distinct_ids
50          50
last_updated_at last_id
2026-09-01T00:35:00Z TX000050
run_id status pages_fetched rows_new
1 RUNNING 1 50

5. Run 2: restart with normal settings; resumes from the checkpoint
| marked 1 earlier run(s) ABANDONED (they were killed mid-run)
| == run 2 starting ==
| resuming from updated_since=2026-09-01T00:35:00Z (last id seen: TX000050)
| page 1: 199 new, 0 changed, 1 unchanged, 0 rejected, has_more=True
| page 2: 200 new, 0 changed, 0 unchanged, 0 rejected, has_more=True
| page 3: 200 new, 0 changed, 0 unchanged, 0 rejected, has_more=True
| page 4: 199 new, 0 changed, 0 unchanged, 1 rejected, has_more=True
| page 5: 151 new, 0 changed, 0 unchanged, 0 rejected, has_more=False
| == run 2 COMPLETED: 5 pages, 949 new, 0 changed, 1 unchanged, 1 rejected (429s: 0, 500s: 0) ==
rows_loaded distinct_ids
999          999
record_id reason run_id last_seen_run_id
TX000777 missing player_id 2 2
id amount
TX000500 250.00

6. Check every record against the API
| [429 rate limited] sleeping 1.0s (attempt 1)
| API records (unique ids): 1,000
| loaded in transactions: 999
| quarantined in ingest_rejects: 1 ['TX000777']
| accounted for (loaded + quarantined): 1,000
| missing: 0 []
| unexpected: 0 []
| stale (change not loaded): 0 []
| duplicate rows: 0
| VERIFIED: the table matches the API exactly

```

demo.py, steps 3-6: hard kill mid-page, restart from the checkpoint, check every id against the API.

Run 3, straight after, finds nothing new or changed. After the interviewer's step (POST /admin/advance), run 4 loads exactly the 25 new and 40 changed records. `verify_against_api.py` then reads the whole API and compares id by id: 1,025 ids = 1,024 loaded + 1 quarantined (TX000777, missing player_id), with 0 missing, 0 stale versions and 0 duplicates. A live 500 and a 429 were retried along the way.

```

Exercise 2: new provider activity, incremental rerun, final state

$ python demo.py (steps 7-10: rerun, new provider activity, rerun, final state)
7. Run 3: run again straight away (nothing has changed at the provider)
| == run 3 starting ==
| resuming from updated_since=2026-09-01T11:40:00Z (last id seen: TX001000)
| [500 server error] backing off 2s (attempt 1)
| page 1: 0 new, 0 changed, 1 unchanged, 0 rejected, has_more=False
| == run 3 COMPLETED: 1 pages, 0 new, 0 changed, 1 unchanged, 0 rejected (429s: 0, 500s: 1) ==

8. Interviewer step: POST /admin/advance (new activity at the provider)
{"ok": true, "total_records": 1025}

9. Run 4: loads only the new and changed records
| == run 4 starting ==
| resuming from updated_since=2026-09-01T11:40:00Z (last id seen: TX001000)
| page 1: 25 new, 40 changed, 1 unchanged, 0 rejected, has_more=False
| == run 4 COMPLETED: 1 pages, 25 new, 40 changed, 1 unchanged, 0 rejected (429s: 0, 500s: 0) ==
| [429 rate limited] sleeping 1.0s (attempt 1)
| API records (unique ids): 1,025
| loaded in transactions: 1,024
| quarantined in ingest_rejects: 1 ['TX000777']
| accounted for (loaded + quarantined): 1,025
| missing: 0 []
| unexpected: 0 []
| stale (change not loaded): 0 []
| duplicate rows: 0
| VERIFIED: the table matches the API exactly

10. Final state
rows_loaded distinct_ids duplicates
1024 1024 0
run_id status pages_fetched rows_new rows_changed rows_unchanged rows_rejected rate_limit_hits server_error_hits seconds
1 ABANDONED 1 50 0 0 0 0 0 0
2 COMPLETED 5 949 0 1 0 0 0 0
3 COMPLETED 1 0 0 1 0 0 1 2
4 COMPLETED 1 25 40 1 0 0 0 0
last_updated_at last_id
2026-09-04T00:16:00Z TX001024

RESULT: PASS: after a hard kill, a restart and new provider activity, the table matches the API exactly: no duplicates, nothing missing, every change
loaded, bad record quarantined.

```

demo.py, steps 7-10: rerun, new provider activity, incremental rerun, final state.

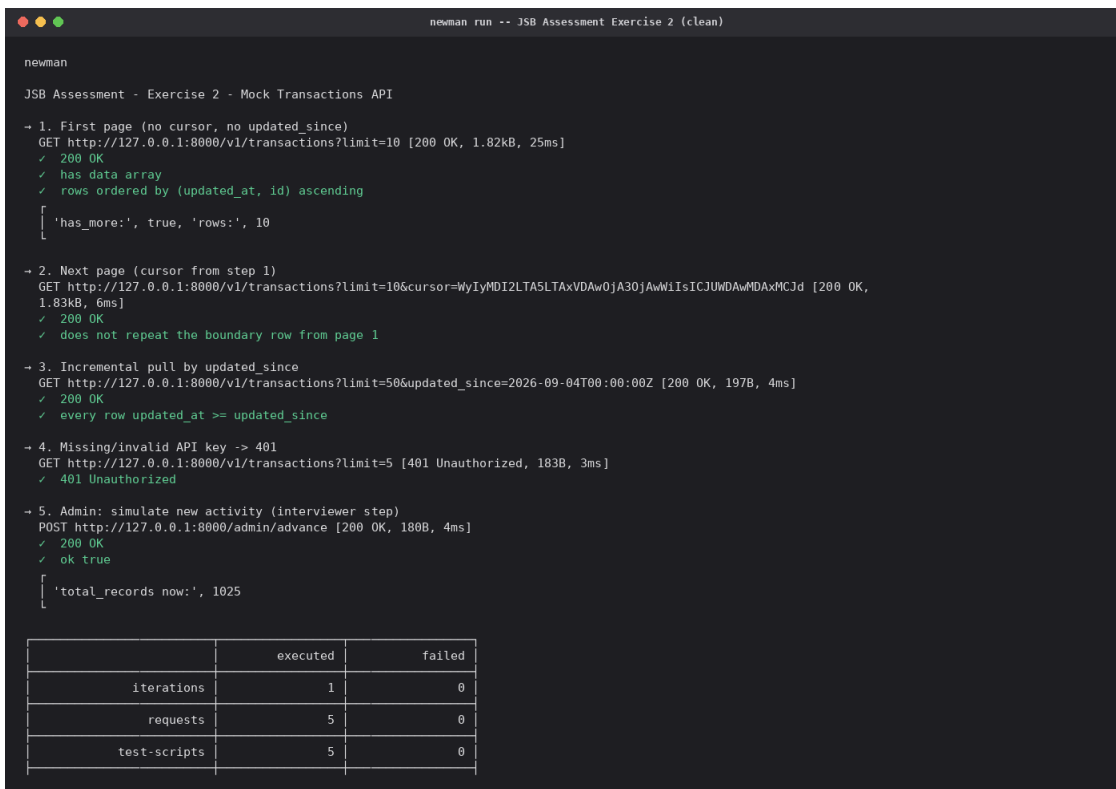
The load continues incrementally into the reporting layer: the dbt model `fct_api_transactions` picks up only rows the ingestion stamped since its last run. `incremental_demo.py` shows 999 rows on the first build, 0 on a rerun, and exactly the 65 new and changed rows after the new activity, with the mart then equal to the source.

Checks and tests

- `checks.sql`: duplicates, reject accounting, run history, stale or failed runs, and the last successful position.
- `verify_against_api.py`: every id in the table against the API (missing, unexpected, stale, duplicates); exits 1 on any mismatch.
- 9 unit tests with no network or database: exit codes (401, exhausted 429 and 500 retries, success), the run lock, validation of unclean records, and the new / changed / unchanged classification.

API contract — Postman

A Postman collection documents the API's pagination, filtering, auth and admin-simulation behaviour, and includes a self-paginating request that walks every page and reports the total record count.



```

newman

JSB Assessment - Exercise 2 - Mock Transactions API

→ 1. First page (no cursor, no updated_since)
GET http://127.0.0.1:8000/v1/transactions?limit=10 [200 OK, 1.82kB, 25ms]
✓ 200 OK
✓ has data array
✓ rows ordered by (updated_at, id) ascending
[
  'has_more:', true, 'rows:', 10
]

→ 2. Next page (cursor from step 1)
GET http://127.0.0.1:8000/v1/transactions?limit=10&cursor=MyIyMDI2LTA5LTxVDAwOjA3OjAwWiIsICJUMDAwMDAxMCJd [200 OK, 1.83kB, 6ms]
✓ 200 OK
✓ does not repeat the boundary row from page 1

→ 3. Incremental pull by updated_since
GET http://127.0.0.1:8000/v1/transactions?limit=50&updated_since=2026-09-04T00:00:00Z [200 OK, 197B, 4ms]
✓ 200 OK
✓ every row updated_at >= updated_since

→ 4. Missing/invalid API key -> 401
GET http://127.0.0.1:8000/v1/transactions?limit=5 [401 Unauthorized, 183B, 3ms]
✓ 401 Unauthorized

→ 5. Admin: simulate new activity (interviewer step)
POST http://127.0.0.1:8000/admin/advance [200 OK, 180B, 4ms]
✓ 200 OK
✓ ok true
[
  'total_records now:', 1025
]



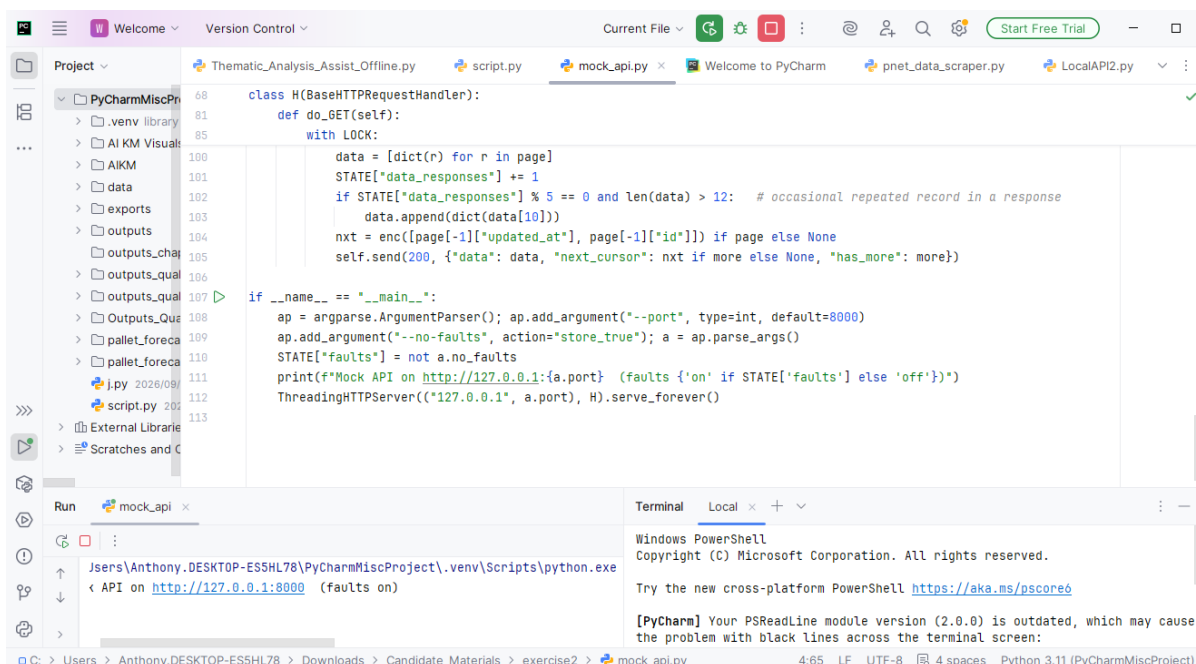
|              | executed | failed |
|--------------|----------|--------|
| iterations   | 1        | 0      |
| requests     | 5        | 0      |
| test-scripts | 5        | 0      |


```

Newman run of the Exercise 2 collection.

Live run — Postman Desktop against the local mock API

The same collection was run in Postman Desktop on a Windows workstation against mock_api.py started from PyCharm, an independent environment from the one the code was built in.



```

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        with LOCK:
            data = [dict(r) for r in page]
            STATE["data_responses"] += 1
            if STATE["data_responses"] % 5 == 0 and len(data) > 12: # occasional repeated record in a response
                data.append(dict(data[10]))
            nxt = enc([page[-1]["updated_at"], page[-1]["id"]]) if page else None
            self.send(200, {"data": data, "next_cursor": nxt if more else None, "has_more": more})

if __name__ == "__main__":
    ap = argparse.ArgumentParser(); ap.add_argument("--port", type=int, default=8000)
    ap.add_argument("--no-faults", action="store_true"); a = ap.parse_args()
    STATE["faults"] = not a.no_faults
    print(f"Mock API on http://127.0.0.1:{a.port} (faults {'on' if STATE['faults'] else 'off'})")
    ThreadingHTTPServer(("127.0.0.1", a.port), H).serve_forever()

```

```

Run mock_api x
Jusers\Anthony.DESKTOP-ES5HL78\PyCharmMiscProject\.venv\Scripts\python.exe
< API on http://127.0.0.1:8000 (faults on)

Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

[PyCharm] Your PSReadLine module version (2.0.0) is outdated, which may cause
the problem with black lines across the terminal screen:

```

mock_api.py running in PyCharm: "Mock API on http://127.0.0.1:8000 (faults on)".

Run order

Run Sequence

Deselect All

Select All

Reset

- ☒ GET 0. Reset record counter (run this first)
- ☒ GET 1. First page (no cursor, no updated_since)
- ☒ GET 2. Next page (cursor from step 1)
- ☒ GET 3. Incremental pull by updated_since
- ☒ GET 4. Missing/invalid API key -> 401
- ☒ POST 5. Admin: simulate new activity (interview)
- ☒ GET 6. Count ALL records (auto-paginates to the end)

Functional

Performance

Choose how to run your collection

☒ Run manually
 ☐ Schedule runs
 ☐ Automate runs via CLI

Run configuration

Iterations

1

Delay

0

ms

Test data file

Select File

Advanced Settings

☒ Persist responses for a session
 ☐ Turn off logs during run
 ☒ Stop run if an error occurs
 ☒ Keep variable values
 ☐ Run collection without using stored cookies
 ☒ Save cookies after collection run

Start run

Collection Runner configuration: all seven requests, one iteration.

JSB Assessment - Exercise 2... - Run results

🔄

▶

+

🔍

▼

Share

🔗

...

Ran today at 04:04:28 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	4s 82ms	18	0	18 ms

All 18

Passed 16

Failed 2

Skipped 0

Errors 0

Console log

List

Grid

PASS

ok true

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200

200 · 8 ms · 31.811 KB 1

PASS page 1: +201 records (running total: 201)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD... 200 · 10 ms · 31.594 KB 1

PASS page 2: +200 records (running total: 401)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD... 200 · 23 ms · 31.625 KB 1

PASS page 3: +200 records (running total: 601)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD... 500 · 9 ms · 190 B 1

PASS page 4: hit 500, retrying same page

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD... 200 · 10 ms · 31.667 KB 1

Runner results, pages 1–4 of the self-paginating count. Page 1 returns 201 rows for limit=200 (an in-page repeated row); page 4 hits a live 500 and retries the same page.

Two assertions fail on request 2 ("Next page"), a one-shot documentation request with no retry logic that happened to land on an injected 429; only the counting request retries. 1,027 rather than 1,025 because the raw count includes the API's in-page repeated rows, which ingest.py collapses.

- **Scheduling:** Airflow or cron every few minutes; the lock already makes overlapping runs harmless.
- **Alerting:** page on a non-zero exit code, a run stuck RUNNING past one interval, a rising reject rate, or checkpoint lag.
- **Secrets:** the API key and database password come from environment variables today; in production, from a secrets manager.
- **Scale:** bulk-load each page into a staging table and MERGE; partition by date; split id ranges across workers if the rate limit becomes the bottleneck.

Area	Tables and key decisions
Bets	One bets header per bet (stake split into real and bonus, plus the grant that paid the bonus part), with a detail table per product: sports_bet_details + bet_legs + sports_events; casino_round_details + games + game_providers; retail_bet_details + retail_locations + devices
Bonuses	bonus_campaigns (trigger, audience, rollover multiple, min odds, max stake, expiry) · player_bonuses (grant, progress, outcome) · bonus_rollover_events (each bet's contribution)

- Money: DECIMAL(18,4), never FLOAT or DOUBLE, which can't hold 0.10 exactly. Odds are DECIMAL(10,3); currency is CHAR(3).
- Time: DATETIME(6), always UTC, in columns named *_at_utc. TIMESTAMP is avoided because it converts through the session time zone. Microseconds keep the order of events within a second.
- Keys: every table has a primary key and every relationship has a foreign key. Natural keys are unique, so a retry can't duplicate: ledger idempotency_key, bet and withdrawal request_id, deposit gateway_ref, one reversal per ledger row.
- 39 CHECK constraints, including: amounts > 0; balance_after = balance_before ± amount; a reversal must point at what it reverses; manual adjustments need a reason; a bonus stake needs its grant; an accumulator has at least 2 legs.
- Indexes on the access paths: ledger (wallet, time) and (player, time) for balances; bets (product, settled time) for NGR; deposits (player, time); bonus links for campaign cost; (player, valid_to) on each history table.

3. How a balance is calculated and guaranteed

A balance is the sum of the ledger up to a moment; wallets.real_balance and bonus_balance are only a cache. Money moves only through two procedures (ledger_posting.sql). post_wallet_txn locks the wallet row, returns the original row if the idempotency key was already posted, refuses an overdraft, writes the ledger row with balance_before and balance_after, and updates the cache, all in one transaction. A reversal (reverse_wallet_txn) is a new, opposite row pointing at the original, which is never edited, and a row can only be reversed once. A correction is a reversal plus the right posting. A dbt test fails the build if any cache differs from its ledger.

```

Exercise 3: test_ledger_posting.py (balance guarantees)

database jsb_platform_ledger_test: ddl.sql + seed.sql + ledger_posting.sql loaded; 18 ledger rows

1. A deposit moves the ledger and the cache together
PASS: ledger row 19: 415.0000 -> 515.0000
PASS: wallet 2 cache is 515.0000

2. Replaying the same event posts nothing
PASS: same key returns row 19; still 19 rows
PASS: balance unchanged by the replay

3. A debit larger than the balance is refused
refused: insufficient balance
PASS: no row posted, balance unchanged

4. A reversal is a new, opposite row; the original is never edited
PASS: row 20: reversal, debit 100.00, points at 19
PASS: row 19 is byte-for-byte unchanged
PASS: wallet 2 back to 415.0000

5. Reversing twice returns the first reversal
PASS: returns row 20; still 20 rows

6. Twenty concurrent postings to one wallet
PASS: all 20 committed (0 errors)
PASS: wallet 1: 890.0000 + 20 x 10.00 = 1090.0000
PASS: no break in any wallet's balance_before -> balance_after chain

7. The CHECK constraints refuse rows that break a rule
PASS: negative amount: refused by ck_wtxn_amount_pos
PASS: balance after that doesn't add up: refused by ck_wtxn_balance_math
PASS: reversal without the row it reverses: refused by ck_wtxn_reversal_link
PASS: manual adjustment without a reason: refused by ck_wtxn_reason
PASS: bonus-funded bet without its grant: refused by ck_bets_bonus_funding

8. Every wallet's cache equals its ledger
PASS: no wallet differs from its ledger

ALL PASS (database jsb_platform_ledger_test dropped)

```

test_ledger_posting.py on a throwaway database: replay, overdraft, reversal, 20 concurrent postings and the CHECK constraints. All pass.

4-5. History, and protecting personal information

- History: VIP tier, tags, and account status + KYC are SCD Type 2 tables. A change closes the current row and opens a new one, so "what tier was this player on 3 September?" and "was this player self-excluded when the bet was placed?" have answers. Money history is the ledger itself; bonus progress is event-sourced.
- Personal data: only player_identity holds names, date of birth, ID number and contact details. Everything else uses the surrogate player_id, and the reporting layer never reads the identity table. Access is one restricted, logged role; the ID number is encrypted in the application with a KMS key; erasure anonymises the identity row and leaves the financial history intact.

6. The four questions — verified output

Definitions: GGR = stakes – payouts on bets settled in the month. Bonus cost = the bonus money wagered on those bets, so NGR = GGR – bonus cost. An unwagered bonus that expires or is forfeited costs nothing; the unwagered part of an active bonus is a liability. No levy is modelled, because the brief doesn't give one.

```

Exercise 3: example_queries.sql against the seed data (MariaDB)

-- example_queries.sql against seed.sql, MariaDB 10.11.14-MariaDB-0ubuntu0.24.04.1

(a) NGR by product for a month
+-----+-----+-----+-----+-----+
| product | turnover | payouts | ggr | bonus_cost | ngr |
+-----+-----+-----+-----+-----+
| sportsbook | 250.0000 | 360.0000 | -110.0000 | 40.0000 | -150.0000 |
| casino | 350.0000 | 80.0000 | 270.0000 | 0.0000 | 270.0000 |
| retail | 100.0000 | 0.0000 | 100.0000 | 0.0000 | 100.0000 |
+-----+-----+-----+-----+-----+

(b) Bonus cost as a % of NGR, by campaign
+-----+-----+-----+-----+
| campaign | bonus_cost | total_ngr | bonus_cost_pct_of_ngr |
+-----+-----+-----+-----+
| Registration Bonus | 40.0000 | 220.0000 | 18.18 |
+-----+-----+-----+-----+

(c) A player's balance at a given date and time
+-----+-----+
| balance_type | balance_as_of |
+-----+-----+
| real | 1160.0000 |
+-----+-----+

+-----+-----+-----+-----+
| as_of | balance_type | balance_as_of |
+-----+-----+-----+-----+
| 2026-09-12 09:15:00.000000 | real | 450.0000 |
| 2026-09-12 10:00:00.000000 | real | 415.0000 |
+-----+-----+-----+-----+

(d) Deposits that failed, and the player later succeeded
+-----+-----+-----+-----+-----+
| player_id | failed_attempt_id | failed_at | succeeded_attempt_id | succeeded_at | minutes_to_success | succeeded_amount |
+-----+-----+-----+-----+-----+
| 2 | 1 | 2026-09-10 09:00:00.000000 | 2 | 2026-09-10 09:04:00.000000 | 4 | 500.0000 |
+-----+-----+-----+-----+-----+

(ledger chain check: 0 rows, no break in any wallet's balance_before -> balance_after chain)

```

example_queries.sql against the seed data (MariaDB 10.11).

Query	Result (September 2026, NAD)
(a) NGR by product for a month	casino 270.00 · retail 100.00 · sportsbook –150.00 (GGR –110.00 less 40.00 bonus cost) · total 220.00
(b) Bonus cost as % of NGR, by campaign	Registration Bonus: 40.00 = 18.18% of NGR. Bonus stakes are traced to the grant that paid them, and the grant to its campaign
(c) A player's balance at a date and time	Player 1 at 2026-09-06 00:00: 1,160.00. Player 2: 450.00 while a wrong credit stood, 415.00 after its reversal and the correct posting
(d) Deposits that failed, then succeeded	Player 2: failed 09:00, succeeded 09:04, 4 minutes later

The same answers come from the dbt marts, the Power BI expected values and the Databricks notebook.

7. From operational design to a reporting model

The dbt project (dbt_jsb_assessment/) builds a star schema: dim_player, dim_player_vip_tier_scd, dim_campaign and dim_date around three facts, each at one grain. fact_bet has one row per bet, with GGR, bonus cost, NGR and the paying campaign. fact_wallet_transaction has one row per ledger movement, loaded incrementally. fact_bonus_transaction has one row per grant, with its cost and outstanding liability. Two marts answer queries (a) and (b), and personal data never enters the model. The same project also runs Exercise 1's reconciliation as a tested model.

```

dbt build --full-refresh (MariaDB 10.11): 78 of 78 PASS

21:00:21 59 of 78 PASS not_null_fact_wallet_transaction_wallet_txn_id ..... [PASS in 0.08s]
21:00:21 63 of 78 START test not_null_fact_bonus_transaction_player_bonus_id ..... [RUN]
21:00:21 60 of 78 PASS unique_fact_wallet_transaction_idempotency_key ..... [PASS in 0.09s]
21:00:21 64 of 78 START test unique_fact_bonus_transaction_player_bonus_id ..... [RUN]
21:00:21 61 of 78 PASS unique_fact_wallet_transaction_wallet_txn_id ..... [PASS in 0.09s]
21:00:21 65 of 78 START sql table model jsb_platform_marts.mart_recon_bridge ..... [RUN]
21:00:21 63 of 78 PASS not_null_fact_bonus_transaction_player_bonus_id ..... [PASS in 0.09s]
21:00:21 66 of 78 START sql table model jsb_platform_marts.mart_recon_summary_by_category ..... [RUN]
21:00:21 64 of 78 PASS unique_fact_bonus_transaction_player_bonus_id ..... [PASS in 0.09s]
21:00:21 62 of 78 OK created sql table model jsb_platform_marts.fact_bet ..... [SUCCESS 6 in 0.29s]
21:00:21 67 of 78 START test not_null_fact_bet_bet_id ..... [RUN]
21:00:21 68 of 78 START test not_null_fact_bet_player_id ..... [RUN]
21:00:21 65 of 78 OK created sql table model jsb_platform_marts.mart_recon_bridge ..... [SUCCESS 11 in 0.29s]
21:00:21 69 of 78 START test relationships_fact_bet_player_id_player_id_ref_dim_player_ ..... [RUN]
21:00:21 68 of 78 PASS not_null_fact_bet_player_id ..... [PASS in 0.09s]
21:00:21 67 of 78 PASS not_null_fact_bet_bet_id ..... [PASS in 0.10s]
21:00:21 70 of 78 START test unique_fact_bet_bet_id ..... [RUN]
21:00:21 68 of 78 OK created sql table model jsb_platform_marts.mart_recon_summary_by_category ..... [SUCCESS 13 in 0.28s]
21:00:21 71 of 78 START test not_null_mart_recon_bridge_step_order ..... [RUN]
21:00:21 72 of 78 START test unique_mart_recon_bridge_step_order ..... [RUN]
21:00:21 69 of 78 PASS relationships_fact_bet_player_id_player_id_ref_dim_player_ ..... [PASS in 0.10s]
21:00:21 73 of 78 START test not_null_mart_recon_summary_by_category_category ..... [RUN]
21:00:21 70 of 78 PASS unique_fact_bet_bet_id ..... [PASS in 0.10s]
21:00:21 71 of 78 PASS not_null_mart_recon_bridge_step_order ..... [PASS in 0.09s]
21:00:21 74 of 78 START test unique_mart_recon_summary_by_category_category ..... [RUN]
21:00:21 75 of 78 START sql table model jsb_platform_marts.mart_ngr_by_product_monthly ..... [RUN]
21:00:21 72 of 78 PASS unique_mart_recon_bridge_step_order ..... [PASS in 0.09s]
21:00:21 73 of 78 PASS not_null_mart_recon_summary_by_category_category ..... [PASS in 0.08s]
21:00:21 74 of 78 PASS unique_mart_recon_summary_by_category_category ..... [PASS in 0.05s]
21:00:21 75 of 78 OK created sql table model jsb_platform_marts.mart_ngr_by_product_monthly ..... [SUCCESS 3 in 0.10s]
21:00:21 76 of 78 START test unique_mart_ngr_by_product_monthly_concat_product_month_start ..... [RUN]
21:00:22 76 of 78 PASS unique_mart_ngr_by_product_monthly_concat_product_month_start ..... [PASS in 0.02s]
21:00:22 77 of 78 START sql table model jsb_platform_marts.mart_bonus_cost_pct_of_ngr ..... [RUN]
21:00:22 77 of 78 OK created sql table model jsb_platform_marts.mart_bonus_cost_pct_of_ngr ..... [SUCCESS 1 in 0.07s]
21:00:22 78 of 78 START test unique_mart_bonus_cost_pct_of_ngr_concat_campaign_name_month_start ..... [RUN]
21:00:22 78 of 78 PASS unique_mart_bonus_cost_pct_of_ngr_concat_campaign_name_month_start ..... [PASS in 0.02s]
21:00:22
21:00:22 Finished running 11 view models, 11 table models, 54 tests, 2 incremental models in 0 hours 0 minutes and 3.52
seconds (3.52s).
21:00:22
21:00:22 Completed successfully
21:00:22
21:00:22 Done. PASS=78 WARN=0 ERROR=0 SKIP=0 TOTAL=78

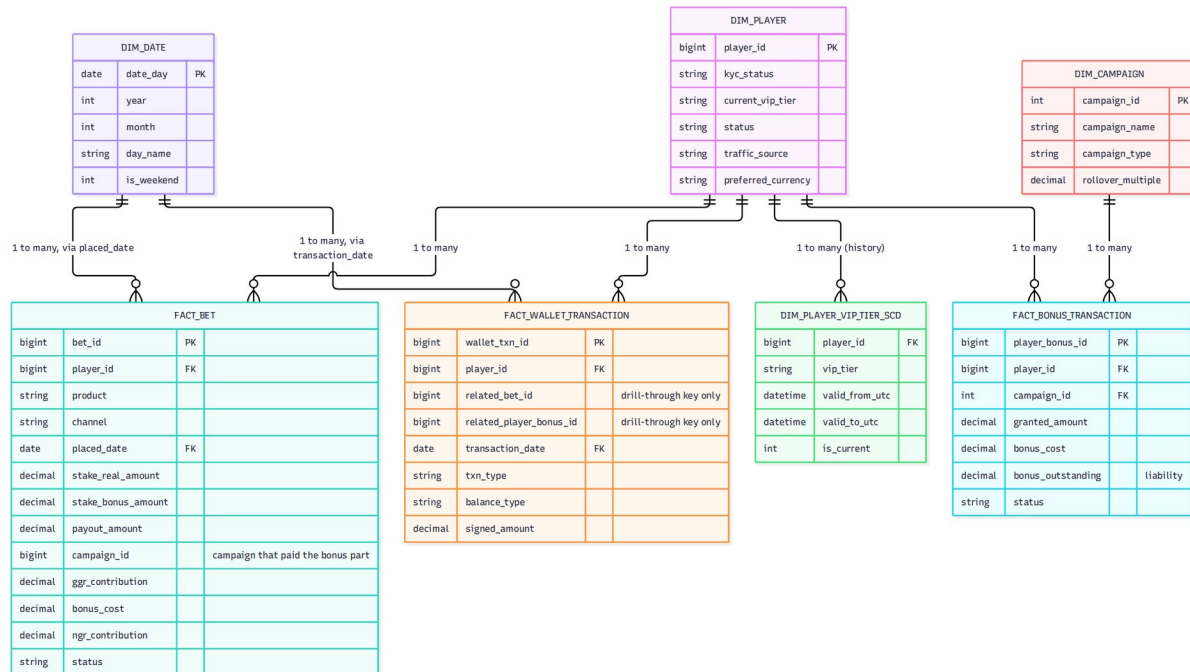
```

dbt build across all three exercises: 78 of 78 pass (24 models, 54 tests), 0 errors. Every mart has a primary key; two models load incrementally.

Power BI — Reporting Data Model and Report

Tools used: Power BI project format (.pbip), generated by a Python script from one set of definitions, with the dbt marts embedded as data.

The dbt marts become a Power BI project, JSB_Assessment.pbip, covering all three exercises. The semantic model has 11 tables, 7 relationships and 28 DAX measures (22 calculations and 6 colour rules for the KPI tiles), and the report has four pages. The data is embedded in the project, so it opens and refreshes on any machine with no folder path or database connection to set up. The same script writes the model, the report pages, the readable DAX file and a list of expected values, then validates them, so the four cannot drift apart.



Power BI model view for Exercise 3: three dimensions filter three facts. Facts are not joined to each other.

Modelling decisions worth calling out

- Facts are not joined to facts. fact_wallet_transaction keeps related_bet_id and related_player_bonus_id as drill-through keys only. Relating them would give dim_player two filter paths to the wallet table. Power BI rejects that as an ambiguous model, and in any tool it is a source of silently wrong numbers. The build script checks every pair of tables and refuses to build if any pair has more than one path.
- Money is fixed decimal (Currency.Type, 4 dp, matching DECIMAL(18,4)), and the CSVs are parsed with en-US culture. A South African regional setting, which uses a comma as the decimal separator, therefore can't misread 200.0000.
- The Exercise 1 and 2 tables (fct_recon_exceptions, mart_recon_bridge, ingest_runs, transactions) stand alone. They share no keys with the player model, so they have no relationships to it, and a filter on one page can't leak into another.

Measures

Measure	Definition
GGR	Stakes – payouts on settled (won/lost) bets
Bonus Cost (realised)	Bonus money wagered on settled bets
NGR	GGR – Bonus Cost (realised) — query (a)
Campaign Bonus Cost / % of NGR	Bonus money wagered from each campaign's grants, ÷ total NGR — query (b)
Bonus Liability Outstanding	Unwagered part of bonuses still active: owed, not yet a cost
Balance as of selected date	Sum of the ledger up to the last date in the date filter — query (c)
Settlements Matched Exactly / Exceptions	Counts of OK rows and of every other category in fct_recon_exceptions
Act Now Value	Impact of the two act-now categories: settled but marked FAILED, and unrecognised settlements
Bridge Amount / Bridge Residual	Sum of the bridge steps; residual = gateway total – steps, which must be 0.00
Transactions Loaded / Runs / Rows Rejected / Retries	Exercise 2's loaded rows and run log

Report pages and expected values

- NGR overview (Ex. 3): GGR, realised bonus cost, NGR and bonus liability cards; GGR vs NGR by product; bonus cost as a percentage of NGR by campaign.
- Player balances (Ex. 3): a date slicer, deposits, and each player's balance rebuilt from the ledger as of the slicer's end date.
- Reconciliation (Ex. 1): headline cards, the bridge as a waterfall from the internal total to the gateway total, exceptions by category, and a detail table.
- Ingestion monitoring (Ex. 2): loaded rows, runs, rejected rows and rate-limit retries; the run history, including the run killed mid-page; transactions by status.

Each value below was computed from the CSVs with pandas, independently of the DAX, and matches the MySQL, MariaDB and dbt results. The full list is in [powerbi/expected_values.md](#).

Visual	Expected value
GGR / Bonus Cost (realised) / NGR / Liability	260.00 / 40.00 / 220.00 / 10.00
NGR by product	casino 270.00 · retail 100.00 · sportsbook –150.00
Registration Bonus: cost / % of NGR	40.00 / 18.18%
Player 1 balance, slicer ending 2026-09-06	1,160.00 (real)
Balances, full date range	Player 1: 890.00 · Player 2: 415.00 · Player 3: 90.00 real + 10.00 bonus
Settlements matched exactly / exceptions	274 / 43
Act Now Value / Bridge Residual	3,150.00 / 0.00
Waterfall	218,280.00 → 217,979.97 in 10 adjustment steps

Visual	Expected value
Transactions loaded / runs / rows rejected / API retries	1,024 / 4 / 1 / 1
Transactions by status	completed 587 · failed 214 · pending 207 · reversed 16

How it was verified

Power BI Desktop doesn't run in the Linux environment this was built in. Instead, the build script checks that every visual field, sort and DAX reference resolves to the model, that every CSV's columns match the model, that every relationship column exists, and that no filter path is ambiguous. A negative test confirmed it rejects an ambiguous relationship and a misspelled measure. The report layout follows the supplied Demo.pbix (Power BI Desktop 2.130, CY24SU06 theme).

Power BI Desktop has since opened an earlier version of this project on a Windows machine: the pages and visuals loaded. The one failure was that the data folder path didn't exist on that machine, and embedding the data removes that step. If Desktop objects to the project for any other reason, powerbi/README.md gives a five-minute manual route to build the same model in Demo.pbix.

Building this layer also caught a real defect upstream. The seed data's wallet balance cache didn't match its own ledger, breaking the rule that the ledger is the truth. The seed was fixed, and a dbt test (assert_wallet_cache_matches_ledger) now fails the build if the two ever diverge. The test failed on the old data and passes on the corrected data.

Databricks — The Same Three Exercises on Delta Lake

Tools used: Databricks Free Edition (serverless compute, Unity Catalog, Delta Lake); open-source Spark 4 + Delta 4 for the local test.

The three exercises also run as Databricks notebooks. `build_notebooks.py` generates them from the project's own files, so they can't drift from the MySQL version: the two Exercise 1 CSVs, the Exercise 2 mock API and the retry and validation code from `ingest.py`, and Exercise 3's `ddl.sql` translated to Delta plus `seed.sql`. Each notebook ends with checks that fail loudly if a figure differs from the MySQL and dbt results.

Notebook	What it does	Checks
01_exercise1_reconciliation	Loads both files with an idempotent MERGE (twice, to prove no duplicates), categorises every row and builds the bridge	17: every category's count and value equal MySQL/dbt; bridge residual 0.00
02_exercise2_incremental_ingestion	Serves the supplied mock API inside the notebook and loads it into Delta with MERGE. Crashes on purpose between a page and its checkpoint, restarts, then loads new activity	6: table equals the API after the restart and after new activity; exactly 25 new and 40 changed; crashed run ABANDONED; one reject
03_exercise3_schema	The 24 tables in Delta with 39 enforced CHECK constraints and Unity Catalog keys, the seed data and the four queries	9: negative stake refused; no duplicate on 44 keys; no orphan on 32 foreign keys; the four answers; cache = ledger

Run on the Databricks workspace

On 2026-09-28, one job ran the three notebooks in order (ex1 → ex2 → ex3) on serverless compute. All three tasks succeeded, and all 32 checks passed, the same total as the local test.

Task	Result	Run id
Job: JSB assessment – all exercises	SUCCESS	460654207301296
ex1 — reconciliation	SUCCESS · 17/17 PASS	453545434689160
ex2 — incremental ingestion	SUCCESS · 6/6 PASS	340425981901042
ex3 — schema design	SUCCESS · 9/9 PASS	685793852884612

Run pages: <https://dbc-ea48b979-9753.cloud.databricks.com/?o=7474649344710062#job/308746372139777/run/<run id>>. The full URLs are in `databricks/evidence/databricks_run.md`.

What Databricks changes, and what it needed

- Keys: Unity Catalog primary and foreign keys are declared but not enforced, and there is no UNIQUE. Loads therefore MERGE on the key, and the notebooks check every key for duplicates and every foreign key for orphans. CHECK and NOT NULL are enforced by Delta, and the notebook proves it by trying a negative stake.
- Restartability: Delta commits one table at a time, so writes are ordered data → rejects → checkpoint → counters, each an idempotent MERGE. A crash re-reads at most one page, and re-applying it changes nothing. Setting the job's Maximum concurrent runs to 1 replaces MySQL's GET_LOCK.

- Two fixes only a real serverless run could show. First, serverless refuses connections to 127.0.0.1 and to fixed ports, so on Databricks the mock API binds an OS-assigned port and is called by host name. Second, Delta requires a generated column's type to match exactly, and DECIMAL(18,4) + DECIMAL(18,4) is DECIMAL(19,4), so generated expressions are now cast to the column type. No check was changed, and the local test still passes.

```

Databricks notebooks on Spark 4 + Delta 4 (run_local.py): 32 of 32 PASS

PASS: 326 deposits and 386 settlements; reloading adds no duplicates
PASS: BREAK: deposit SUCCESS, no gateway settlement found: 5 rows, 2,700.00
PASS: BREAK: duplicate gateway settlement for one reference (double-credit risk): 4 rows, 0.00
PASS: BREAK: duplicate internal SUCCESS deposit for one settlement (double-credit risk): 6 rows, 0.00
PASS: BREAK: net amount is not gross minus fee: 2 rows, 4.00
PASS: BREAK: payment confirmed, wallet not credited: 2 rows, 250.00
PASS: BREAK: settled fee differs from contracted fee: 4 rows, 3.75
PASS: BREAK: settled gross amount differs from internal amount: 4 rows, -900.00
PASS: BREAK: unrecognised settlement (no internal record): 4 rows, 2,900.00
PASS: NOT A PROBLEM: rounding difference <= 1 cent: 3 rows, 0.03
PASS: OK: matched, amount and fee correct: 274 rows, 0.00
PASS: REVERSAL: gateway reversed/charged back after settlement: 3 rows, 850.00
PASS: TIMING: prior-period deposit settled at start of period: 3 rows, 2,600.00
PASS: TIMING: settlement expected in next period (created near cut-off): 3 rows, 3,650.00
PASS: no unexpected categories
PASS: bridge residual is exactly 0.00
PASS: recon_key is unique (one row per deposit/settlement pair)
== 01_exercise1_reconciliation: OK in 44s
PASS: the table matches the API exactly
PASS: the table matches the API exactly
PASS: the crashed run is marked ABANDONED
PASS: a rerun with nothing new loads nothing
PASS: after new activity: exactly 25 new and 40 changed
PASS: the bad record (TX000777) is quarantined once
== 02_exercise2_incremental_ingestion: OK in 82s
PASS: a negative stake violates ck_bets_stake_nonneg and is rejected
PASS: no duplicates on any of the 44 primary and natural keys
PASS: no orphans on any of the 32 foreign keys
PASS: NGR: casino 270, retail 100, sportsbook -150
PASS: Registration Bonus: cost 40.00 = 18.18% of NGR
PASS: player 1 real balance on 2026-09-06 is 1,160.00
PASS: player 2: 450.00 while the wrong credit stood, 415.00 after its reversal and the correct posting
PASS: player 2 failed, then succeeded 4 minutes later
PASS: every wallet's cached balance equals the sum of its ledger
== 03_exercise3_schema: OK in 124s

```

The same notebooks on open-source Spark 4 + Delta 4 (run_local.py): all 32 checks pass.